EHR-Copilot: A Multi-Agent RAG Pipeline for Faithful Clinical Question Answering over Electronic Health Records

A Graduate Project Report
Submitted to
San Jose State University

In Partial Fulfillment of the Requirements for the Degree
Master of Science in Software Engineering

Prepared by:
Author Name

Masters Project
Spring 2026

Preface

Most of what clinicians do, day-to-day, is read. A physician walking into a shift inherits a chart that already runs thousands of tokens long — admission note, problem list, a rolling series of lab panels, medication orders, imaging reports, a dozen progress notes from the teams that came before. The first question the physician asks about a patient is almost never something a textbook can answer. It is specific to that patient's timeline, numbers, and medications. And it has to be answered correctly, because the downstream action — a prescription, a dose change, a consult order — can harm the patient if the answer is wrong.

This report describes a system I built to make that kind of question-answering more trustworthy. The system is called *EHR-Copilot*. It takes a natural-language question about a single patient and an identifier for that patient's FHIR bundle, retrieves the handful of chart fragments that actually bear on the question, drafts an answer, checks that answer against the retrieved evidence with a small pipeline of specialized agents, and returns either a cited answer or an explicit abstention. What it tries not to do is hallucinate, and most of the architectural choices in the system are there because a one-shot large language model, handed the whole chart, does hallucinate.

The work was done as part of my Master of Science in Software Engineering at San Jose State University. It is grounded in a simple wager: that for high-stakes, context-specific question-answering, a small pipeline of cooperating agents — each doing one thing and doing it well — beats a single large model doing everything in one pass.

Acknowledgment

I owe thanks to several people and groups.

My project advisor at San Jose State University provided steady feedback through the full arc of this work, from the first sketch of a router-and-critic loop on a whiteboard to the final evaluation against frontier models. The committee members helped me narrow the scope when I was trying to put too much into a single report.

The MIMIC-IV team at PhysioNet, along with the Synthea project, maintain the data that makes a project like this possible outside a hospital system. The MedHallu benchmark filled a gap I could not have built a training set for on my own. Microsoft Research's PubMedBERT made clinical-domain retrieval start working at something close to the right quality out of the box, and the authors of LoRA and GRPO provided the adaptation and fine-tuning recipes that I leaned on when I ran the research-extension experiments in Chapter 7.

Finally, the Hugging Face, sentence-transformers, LangGraph, FAISS, and Ollama communities deserve credit for the infrastructure I built on top of. It is easy to forget, when everything works, how many pieces had to be in place for the system to run end-to-end on a laptop.

The Designated Advisor Approves the Report Titled

EHR-Copilot: A Multi-Agent RAG Pipeline for Faithful Clinical Question Answering over Electronic Health Records

by

Author Name

APPROVED FOR THE COURSE OF ENGR 295 SPRING 2026

Master's Project

DEPARTMENT OF SOFTWARE ENGINEERING

SAN JOSE STATE UNIVERSITY

Spring 2026

PRIMARY ADVISOR: Prof. Advisor Name
Date: _____

COMMITTEE MEMBERS: _____

COURSE INSTRUCTOR: Prof. Instructor Name
Date: _____

Contents

Abstract

Large language models do well on board-style medical exams and struggle, badly, on patient-specific questions over real charts. When a model is handed a ten-thousand-token FHIR bundle and asked what medications the patient is currently on, the failure modes are not subtle: it confidently names drugs that were discontinued months ago, quotes lab values that do not appear in the chart, or refuses to answer a question that has a perfectly clear structured answer. In the clinical setting that is not acceptable.

This project presents **EHR-Copilot**, a multi-agent Retrieval-Augmented Generation (RAG) pipeline aimed at that gap. The system has five agent stages — a Router that classifies the query, a Retrieval stage that combines dense PubMedBERT embeddings with BM25 sparse matching through Reciprocal Rank Fusion and a cross-encoder reranker, a Reasoning agent that produces a draft answer from the top retrieved chunks, a pair of deterministic validators for temporal ordering and numeric-unit consistency, and a Critic that either approves the draft, sends it back once for revision, or abstains. All evidence used by the draft is mapped back to specific chart chunks so every sentence in the final answer carries a citation a clinician can click through to verify.

Two additional pieces support clinical deployment: a patient-scoped ephemeral FAISS index that materializes per session and self-destructs on logout so cross-patient leakage is impossible by construction, and a SHA-256 hash-chained audit log that records every retrieval, prompt, validator verdict, and citation mapping for each query.

I evaluated the pipeline on eighty clinician-style queries across ten MIMIC-IV patients, with automatic ground truth extracted directly from the patients' FHIR bundles to avoid LLM-as-judge bias. Against five one-shot frontier baselines (GPT-5, Claude Sonnet 4.5, Claude Haiku 3.5, Gemini 3 Pro, and GLM 4.6), the pipeline reaches Entity F1 = 0.639 with precision 0.850 and abstention accuracy 100%. On medication queries — where one-shot models keep confusing completed prescriptions with current ones — the pipeline scores 0.770 compared with 0.007 for GPT-5, the best-known example of where a focused retrieval stage pays off most. On the MiniMax M2.5 backbone specifically, Entity F1 reaches 0.671.

Chapter 7 documents a research-extension track in which I replaced the single Critic with a three-agent Multi-Agent Debate (MAD) layer and trained the debate agents with Group Relative Policy Optimization and counterfactual credit assignment. The debate layer improves Detection F1 from 0.551 to 0.645 on the MedHallu benchmark, and MARL with separate LoRA adapters reaches 0.752 under matched prompts. That track also surfaces a sharp negative result: RL-trained gains mostly vanish when the deployment prompt format differs from the training prompt format, which I argue is a general obstacle for applying RL to multi-stage NLP pipelines.

1 Identification and Significance of the Problem or Opportunity

1.1 Background and Societal Context

Clinical documentation in the United States has been electronic for most of a decade, and the resulting volume is a problem in its own right. A single hospital admission produces dozens of notes, hundreds of discrete lab observations, and a medication list that changes multiple times per day. Clinicians spend a documented fraction of their shift reading, and more and more of it is spent reading their own charts to answer specific questions: "what is this patient's A1c trend over the past year," "are any of the antibiotics on the current med list contraindicated with the renal function we saw this morning," "when was the last cardiology consult and what did they recommend."

Large language models are an appealing interface to that pile of information. They take a natural question and return a natural answer, and on public benchmarks such as MedQA and PubMedQA they perform remarkably well. The problem is that those benchmarks test general medical knowledge, not patient-specific reasoning over a real chart. The moment the question depends on *this* patient's data, the ground shifts.

1.2 Why Hallucinations in Clinical QA Are a Different Problem

A general-purpose LLM hallucinates a citation on a consumer search query and the worst consequence is a broken link. The same hallucination about a patient's current dose of warfarin is a direct patient-safety incident. The asymmetry matters, and it changes what "good enough" means for a model. In a consumer setting we tolerate a few percent error; in a clinical setting, even that rate is too high if the errors land on the wrong cases.

Three failure patterns dominate when one-shot LLMs are asked clinical questions over real charts. The first is confident fabrication: the model generates a plausible clinical sentence that does not appear in the record. The second is temporal confusion: the model treats discontinued medications or completed encounters as if they were current. The third is numeric drift: the model quotes a lab value that is either subtly wrong or attached to the wrong unit. These are not random errors. They correlate with structural features of the chart — length, redundancy, and the presence of `status: completed` fields that a model interprets differently than a clinician would.

1.3 Gaps in Current Clinical RAG Systems

Retrieval-Augmented Generation is the obvious response. Instead of feeding the whole chart to the model, retrieve a small number of relevant chunks, then generate the answer grounded in those

chunks. In practice, a naive single-pass RAG does not close the gap, for three reasons worth spelling out.

First, clinical queries are not one thing. A medication reconciliation question needs exact keyword matching over structured resource types. A temporal question needs chronological reasoning. A numeric lab question needs unit conversion and arithmetic. A single retrieval strategy optimized for an "average" query underperforms every specialized configuration.

Second, a single decoder pass cannot simultaneously retrieve, reason, and verify. The model writes its draft and it believes the draft. Verification has to be a separate stage operated by a separate component, or it does not catch the errors that matter.

Third, the verification stage itself, when it exists in published work, is usually a single critic giving a global thumbs-up. That misses the subtle cases. If three of the four claims in the answer are supported and the fourth is fabricated, a global critic often approves. A claim-level critic with an abstention policy does not.

1.4 Why a Specialized Multi-Agent Pipeline

The thesis of this project is narrow: for high-stakes, patient-specific clinical QA, the right architecture is a short pipeline of specialized agents, each doing one thing, rather than a single powerful model doing everything. Specialized agents let us make the retrieval layer actually retrieve, the reasoning layer actually reason over a focused context, and the verification layer actually verify claim by claim. The cost is latency — five or six serial LLM calls instead of one — and the benefit is that the pipeline can say "I don't know" in a principled way when the evidence does not support an answer.

1.5 Project Objectives and Significance

Concretely, the objectives of the project are:

- Build a five-stage RAG pipeline (Router, Retrieval, Reasoning, Validators, Critic) that operates over FHIR-formatted patient data and produces cited, verifiable answers or explicit abstentions.
- Beat strong one-shot frontier LLMs on clinician-style queries extracted from real chart data, measured with automatic entity-level ground truth.
- Make privacy a first-class architectural constraint: the patient index is built in memory, scoped to a single session, and destroyed on logout.
- Make trust auditable: produce a hash-chained audit trail that a compliance team can inspect per query.

- Explore, as a research extension, whether a structured multi-agent debate layer and reinforcement learning can push hallucination detection further on standard benchmarks, and honestly document where that extension works and where it does not.

1.5.1 Relationship to Existing Research

The project sits at the intersection of a few threads. From biomedical RAG (Almanac, CRAG) I took the principle that retrieval quality is worth fine-tuning for separately. From multi-agent LLM frameworks (AutoGen, MedAgents) I took the idea that specialized roles outperform monolithic prompting. From multi-agent debate work (Du et al., Liang et al.) I took the adversarial verification structure explored in the research extension. From the GRPO algorithm introduced in DeepSeek-R1, and its medical extension HuatuoGPT-o1, I took the reward-model-free RL recipe that makes fine-tuning small agents tractable on academic compute. None of these pieces is new on its own. The contribution of this project is the way they are combined and the honest evaluation against strong modern baselines.

2 Identify Assumptions

Every engineering project stands on assumptions. The ones that matter most for EHR-Copilot concern the quality of the underlying data, the behavior of the models in the pipeline, the willingness of a clinical team to adopt an AI assistant in any form, and the legal frame around PHI.

2.1 Availability and Accuracy of EHR Data

The pipeline assumes the patient record arrives in a FHIR-compliant form with the fields a clinical question would turn on: medications (with status and date ranges), observations (labs with units), conditions, encounters, procedures, and diagnostic reports. In development, that was the MIMIC-IV Clinical Database Demo on FHIR, plus synthetic Synthea bundles to stress the ingestion path. In a real deployment, it would be whatever the institution's EHR vendor exposes through its FHIR API. The assumption is that those fields are populated consistently — a brittle assumption in some settings, and one I do not claim to have solved.

2.2 Retrieval Quality Carries the System

A large fraction of the pipeline's quality comes from the retrieval stage. The assumption is that combining domain-tuned PubMedBERT embeddings, BM25 sparse matching, Reciprocal Rank Fusion, and a cross-encoder reranker produces a short list (eight chunks, in practice) that contains the evidence the Reasoning agent needs. When that assumption breaks, the Reasoning agent either hedges or the Critic abstains, which is a graceful failure mode but still a failure.

2.3 Deterministic Validators Catch Structural Errors

The temporal and numeric validators are deterministic — no LLM call — and the pipeline assumes that most of the structurally checkable errors in the draft answer (wrong date ordering, impossible dates, unit mismatches, obvious arithmetic errors) can in fact be caught with careful parsing and unit-aware comparison. Semantic hallucinations that do not trip a structural check fall to the Critic.

2.4 The Critic's Abstention Policy Is Acceptable to Clinicians

The Critic defaults to abstention on uncertainty. Every design choice around it is tilted in that direction: a "when in doubt, flag" verification prompt, a routing policy where any material claim scored zero forces the whole answer to `ABSTAINED`, a hard cap of one revision cycle. This is a deliberate tradeoff, and it assumes clinicians prefer an assistant that occasionally refuses to a confident assistant that is sometimes wrong. In user-research terms that is not yet proven for this

system — it is an assumption I took from published work on clinical decision support and from how radiologists react to overconfident triage tools.

2.5 Privacy by Architecture, Not Only by Policy

The pipeline assumes that the right way to protect PHI is to hold patient data in memory for the duration of a session and destroy it on logout, rather than storing it in a shared index. That requires that session lifecycles are short relative to the sensitivity of the data, and that the index registry correctly frees memory on termination — which I tested but did not formally verify.

2.6 HIPAA, the EU AI Act, and Model Data Residency

The system is designed so that inference runs locally, against an Ollama-hosted LLM, with no data leaving the host. That makes HIPAA and EU AI Act compliance a matter of properly configuring the host environment rather than a matter of vendor trust. The assumption is that institutions able to run a GPU-backed service can accept this deployment model. An institution that cannot run local inference would need a different, vendor-BAA-covered configuration.

2.7 Compute Adequacy

Development ran against a single GPU host for the retrieval service and the local LLM. A production deployment sized for a chart-review workflow — not real-time bedside — is compatible with moderate GPU hardware. Real-time bedside use is explicitly out of scope; latency is discussed in Chapter 9.

3 Feasibility Analysis

3.1 Technical Feasibility

Every component in the pipeline is built on mature, openly available tooling: sentence-transformers for embeddings, NeuML/pubmedbert-base as the base encoder, FAISS for the dense index, BM25S for sparse matching, `cross-encoder/ms-marco-MiniLM-L-6-v2` for reranking, `fhir.resources` and `medspacy` for ingestion, `python-dateutil` and `pint`/`ucumvert` for the deterministic validators, FastAPI for the REST surface, Streamlit for the UI, and Ollama as the local LLM runtime. The integration risk is not in any one component; it is in the contracts between them — the schema of the retrieval chunks, the structure of the audit log entries, the JSON envelope for validator verdicts. Most of the engineering time went there.

3.2 Operational Feasibility

Operationally, the pipeline is a few stateless services plus a session-scoped index registry. The LLM layer is the one stateful piece, and Ollama handles that. Failure modes are well-defined: retrieval timeout abstains, validator exception abstains, Critic timeout abstains. Nothing best-effort-completes when something breaks.

3.3 Economic Feasibility

The cost model is dominated by serial LLM calls (one at Router, one at Reasoning, one at Critic, and sometimes a revision pass). With MiniMax M2.5 pricing at roughly \$0.30/\$1.20 per million input/output tokens, a typical query lands at around one cent. The retrieval and validator stages are effectively free per query; they cost GPU memory for the embedding service and nothing else.

3.4 Social and Clinical Feasibility

EHR-Copilot is positioned as an assistive tool for chart review and documentation support, not a diagnostic or prescriptive tool. That framing matters for clinical acceptability. A clinician who already reads the chart carefully is not replaced by this pipeline; they are given a faster path to the three chunks of the chart that actually bear on their question, with citations they can verify in one click.

3.5 Legal and Ethical Feasibility

Legal considerations are straightforward on paper and harder in practice: HIPAA for PHI handling, the EU AI Act for accountability and transparency if the system is deployed in Europe, and any

state-level privacy rules. The hash-chained audit log addresses the transparency requirement by producing a per-query record that is tamper-evident. The abstention policy addresses the "I do not know" requirement. Role-based access control and the ephemeral index handle the data-residency requirement on the storage side.

4 Project Concepts and High-Level Architecture

4.1 Project Concept

EHR-Copilot is a multi-agent RAG pipeline for patient-specific clinical question answering over FHIR-formatted EHR data. A clinician types a question, picks a patient, and gets back either a short answer with per-sentence citations and a confidence score, or an abstention with an explanation. Between the question and the answer sit five agent stages, a hybrid retrieval engine, a pair of deterministic validators, a citation engine, and an audit logger.

4.2 High-Level System Architecture

At the whiteboard level the pipeline is short:

Listing 4.1: Runtime path of a single query.

```
Query -> Router Agent -> Hybrid Retrieval -> Reasoning Agent ->
    [Temporal Validator || Numeric Validator] ->
    Critic Agent -> (APPROVE | REVISE x1 | ABSTAIN) ->
    Citation Engine -> Answer + [1][2][3]
```

[Figure 4.1 – Master architecture diagram]
Export the mermaid diagram from `ehr-copilot-master-diagram.html` to PNG/PDF, drop it in `figures/architecture.png`, then replace this `fbox` with `\includegraphics[width=\textwidth]{figures/architecture.png}`.

Figure 4.1: EHR-Copilot master architecture.

4.2.1 Data Layer

The data layer ingests FHIR R4 bundles and free-text notes, parses them with `fhir.resources` (Pydantic v2 models), segments notes with `medspacy` into clinical sections (HPI, Assessment/Plan, Labs, Meds, ROS, Physical Exam), normalizes units and dates with `ftfy` and `python-dateutil`, and emits semantically coherent chunks of at most 256 tokens with a 32-token overlap. Each chunk

carries its provenance — which resource type it came from, which note, which section, which time window — which the citation engine uses later.

4.2.2 Retrieval Layer (Patient-Scoped Ephemeral Index)

Every session builds its own index. When a clinician opens a patient, the ingestion pipeline runs, embeddings are computed with `NeuML/pubmedbert-base-embeddings` (768 dimensions), and two indexes materialize in memory: a FAISS `IndexFlatIP` for dense retrieval and a BM25S sparse index for keyword matching. A Hybrid Retriever combines the two with Reciprocal Rank Fusion (k=60), takes the top thirty candidates, and passes them to a cross-encoder reranker (`ms-marco-MiniLM-L-6-v2`) that produces the final top-eight. On session end, an Index Registry destroys the indexes. Nothing persists, and cross-patient leakage is impossible by construction because no two sessions ever share memory.

4.2.3 Reasoning Layer (Multi-Agent Pipeline)

Five agent stages run in sequence:

Router Agent

Classifies the query into one of several types — `TEMPORAL`, `NUMERIC`, `TEMPORAL_NUMERIC`, `FACTUAL`, `MEDICATION`, `SUMMARY`, `REASONING`. One LLM call, around 200 ms.

Retrieval Agent

Runs the hybrid retrieval, performs sub-query expansion for complex queries, and applies temporal filtering when the Router flagged a time window. Zero LLM calls, around 500 ms for the encoding path.

Reasoning Agent

Produces a draft answer from the top-eight chunks, using a chain-of-thought template sized to the query type. One LLM call, around 1.5 s.

Validators (parallel)

A Temporal Validator parses dates in the draft with `python-dateutil` and cross-checks them against a patient timeline it built from the FHIR `Encounter` and `Observation` resources. A Numeric Validator parses units with `pint` / `ucumvert` (UCUM standard) and verifies that quoted values, unit conversions, and any arithmetic the draft performs are correct. Both are deterministic, around 150 ms each, and run in parallel.

Critic Agent

Sees the draft, the evidence chunks, and the validator verdicts, and issues one of three decisions. `APPROVE` if every material claim is supported and the validators pass. `REVISE` once, back to the Reasoning agent, if some claims are unsupported but the evidence is present. `ABSTAIN` if the evidence is insufficient. One LLM call, around 500 ms.

4.2.4 Trust and Compliance Layer

Two supporting systems run alongside the pipeline. The **Citation Engine** splits the approved answer into sentences (`spacy en_core_web_sm`), maps each sentence to its best-matching source chunk with `rapidfuzz` and embedding cosine similarity, inserts `[1]` `[2]` `[3]` markers, and emits an EvidencePack containing the cited text, document IDs, character offsets, and retrieval scores. The **Audit System** writes an append-only, SHA-256 hash-chained record to SQLite (via `aiosqlite`) for every query: the query text, the classification result, the retrieved chunks and their scores, every LLM prompt and completion, the validator verdicts, the Critic's decision, the citation mappings, and the final response. An Integrity Verifier can walk the chain on demand — a `GET /audit/{session_id}` endpoint returns the chain with hash verification — which is what the compliance story ultimately rests on.

4.2.5 Interface Layer

The UI is a Streamlit application for development and demos (chat-style interaction, click-to-verify citation links, a per-query timeline view). The programmatic surface is a FastAPI service with `POST /query`, `POST /patient/load`, `GET /audit/{id}`, and `GET /health` endpoints. Middleware enforces a Request-ID, per-query timing, and patient-scope: the currently loaded patient is pinned to the session, and a query against a different patient is rejected.

5 Phase I Technical Objectives

5.1 Overview

Phase I had two goals. Build the five-stage pipeline end-to-end, and evaluate it honestly against frontier LLMs on questions drawn from real chart data. Everything in Chapter 5 and Chapter 6 is Phase I. The debate and RL work in Chapter 7 is Phase II.

5.2 Dataset Design and Feature Breakdown

5.2.1 Patient Data (MIMIC-IV FHIR + Synthea)

The main evaluation uses the MIMIC-IV Clinical Database Demo on FHIR, which is a de-identified public dataset with structured resources (Patient, Encounter, Observation, Condition, MedicationRequest, MedicationAdministration, Procedure, DiagnosticReport) plus free-text discharge notes. Ten patients were selected to cover the major care patterns (cardiac, respiratory, renal, mixed). For development and stress-testing the ingestion path I also used Synthea-generated synthetic FHIR R4 bundles, because their structure is controllable and they surface edge cases in the FHIR parser that MIMIC does not.

5.2.2 Query Design

Eight queries per patient, eighty total. The queries were written by hand to mirror what a clinician actually asks during chart review, covering each of the Router's query types. Example shapes:

- *Medication.* "What anticoagulants is this patient currently on, and at what dose?"
- *Temporal.* "When was the most recent echocardiogram, and what was the ejection fraction?"
- *Numeric.* "What is the A1c trend over the past twelve months?"
- *Factual.* "What chronic conditions are on this patient's problem list?"
- *Summary.* "Give me a one-paragraph summary of the most recent admission."

5.2.3 Ground Truth Without LLM-as-Judge

Entity-level ground truth was extracted directly from the FHIR resources for each question — for a medication query, the set of active `MedicationRequest.medicationCodeableConcept` codes; for an A1c trend query, the ordered list of A1c `Observation` values with their dates. That gives an evaluation signal that is not biased by a judging LLM and not dependent on paying annotators. It also limits what can be evaluated — a "summary" query needs a softer metric, which is why ROUGE-L and semantic similarity are reported alongside entity F1.

5.2.4 Retrieval Fine-Tuning Triplets (Phase II)

For the Phase II retrieval-fine-tuning experiments, 760 clinical query–chunk triplets were derived from pipeline evaluation logs and used with MultipleNegativesRankingLoss. An earlier attempt with TripletLoss collapsed to near-random accuracy because the positives and hard negatives were too semantically close for a margin loss to separate them.

5.3 Machine Learning and Retrieval

5.3.1 Embedding Model

The base encoder is `NeuML/pubmedbert-base-embeddings` at 768 dimensions. PubMedBERT was pretrained on biomedical literature, and empirically it matches clinical terminology (for example, identifying "hypertension" and "high blood pressure" as near-identical) well enough that the un-fine-tuned model already makes the pipeline usable. Fine-tuning in Phase II (§7.5) raises retrieval MRR from 0.623 to 0.914 on the held-out set, which is the bulk of the improvement available from retrieval-only changes.

5.3.2 Dense + Sparse + Rerank

The Hybrid Retriever combines FAISS dense retrieval (`IndexFlatIP`, inner product, in-memory) with BM25S sparse retrieval (SciPy CSR under the hood), fuses the two rankings with RRF at $k = 60$, takes the top thirty candidates, and reranks them with `ms-marco-MiniLM-L-6-v2` down to the final top-eight chunks that the Reasoning agent actually sees.

5.3.3 Routing

The Router's job is to pick the reasoning prompt template and, when applicable, a retrieval filter. For temporal queries the retriever narrows to chunks in the relevant time window; for numeric queries it prioritizes chunks containing labs and observations; for medication queries it prioritizes `MedicationRequest` resources.

5.4 Agent Pipeline

The five agent stages were described in §4.2.3. What is worth adding here is that each agent has a separate prompt template (stored as Jinja2 files) and each has its own JSON schema for output, validated with Pydantic. A malformed JSON response triggers a single automatic retry; a second malformed response fails closed to `ABSTAIN`.

5.5 API and Interface Layer

5.5.1 FastAPI Backend

The backend exposes `POST /query` (submit a question), `POST /patient/load` (ingest a FHIR bundle and build the ephemeral index), `GET /audit/{session_id}` (return the hash-chained audit trail with integrity verification), and `GET /health` (liveness probe). Patient scope is enforced in middleware: every query is tagged with the session's patient ID, and attempting to query a different patient from within the same session returns a 403.

5.5.2 Streamlit Frontend

The UI shows the answer with inline `[1]` `[2]` `[3]` markers, a drawer that opens the underlying chunk on hover, a per-query timeline view, and a confidence badge. Abstentions surface a dedicated banner with the Critic's reason — "evidence insufficient," "unit mismatch in claimed lab value," and so on — rather than a blank answer.

6 Phase I Statement of Work

6.1 Overview

Phase I deliverables: the ingestion pipeline, the ephemeral index, the five-stage agent pipeline, the citation engine, the audit system, the FastAPI surface, the Streamlit UI, and the Track 1 evaluation described in Chapter 9.

6.2 Data Preparation

FHIR bundles are parsed into Pydantic models, notes are segmented with `medspacy`, encodings and dates are normalized, and chunks are emitted at 256 tokens with a 32-token overlap. The chunker is section-aware: it will not split across clinical section boundaries (HPI, A/P, Labs, Meds), because a mid-section split tends to produce chunks that look coherent but are missing context.

6.3 Index Construction

Dense embeddings go into FAISS; sparse term frequencies go into BM25S; the indexes live in the Index Registry keyed by `(session_id, patient_id)`. Session timeouts destroy the indexes; explicit logout destroys them; process shutdown destroys them. There is no code path that persists the index to disk.

6.4 Agent Implementation

Each agent is a class with a `run(state) -> state` signature, orchestrated through a small graph runner (LangGraph-style). The state object carries the query, the retrieved chunks, the draft, the validator verdicts, the Critic's decision, and the citation mappings. The Critic's decision determines the outgoing edge: `APPROVE` routes to the citation engine, `REVISE` routes back to the Reasoning agent once (and only once), `ABSTAIN` routes to the abstention path.

6.5 Integration with the Local LLM Layer

All LLM calls go through Ollama. The primary backbone during evaluation was MiniMax M2.5; a local fallback of `qwen2.5:7b-instruct` (128K context, roughly 8 GB VRAM at Q8 quantization) is supported for offline demos. The Prompt Engine is Jinja2-templated with token-budget accounting, and the Response Parser combines JSON-mode output with a regex fallback and a final Pydantic validation pass.

6.6 Audit Log and Citation Engine

Every query emits between six and eight log entries — query received, classification result, retrieved chunks with scores, LLM prompts and outputs per agent, validator verdicts, Critic decision, citation mappings, final response. Each entry includes the SHA-256 hash of the previous entry, so the chain can be verified top-to-bottom on demand.

6.7 Testing and Handoff

End-to-end tests cover the seven query types, the three Critic decisions, and the standard failure paths (retrieval timeout, LLM outage, empty evidence, malformed JSON). Phase I closes with documentation, a reproducible evaluation harness, and a pinned environment lockfile.

6.8 Phase I Timeline (Gantt Chart)

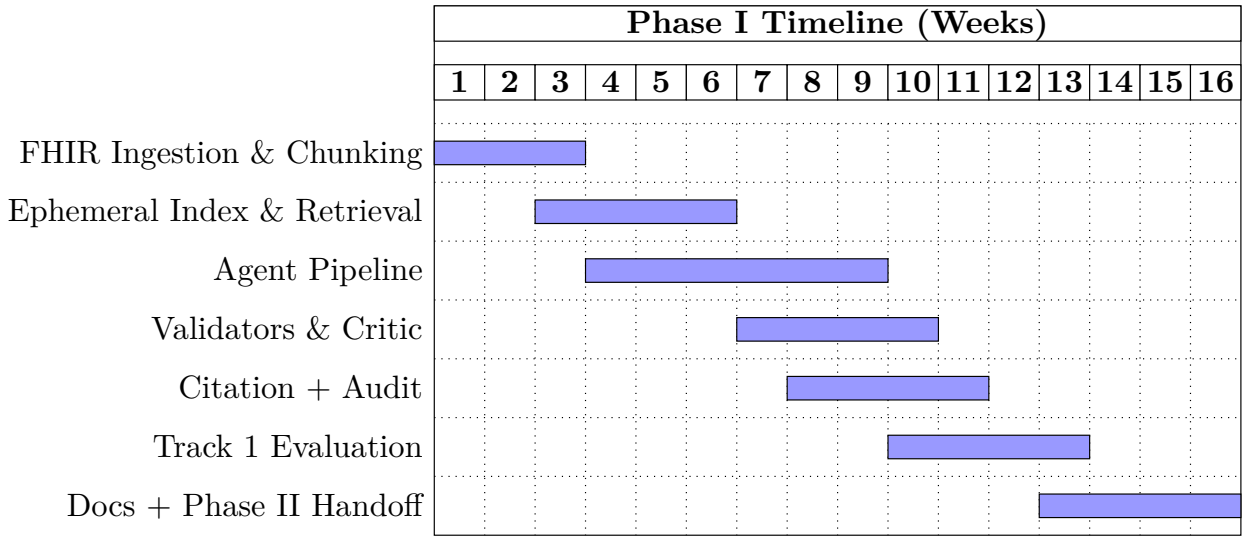

Figure 6.1: Phase I timeline.

7 Phase II Experimental Approach

7.1 Overview

Phase II is a research-extension track. It asks whether a more elaborate verification layer — a three-agent Multi-Agent Debate (MAD) structure, trained cooperatively with reinforcement learning — improves hallucination detection beyond the single-Critic approach used in Phase I. The Phase II experiments run on the MedHallu hallucination-detection benchmark rather than on the patient-level queries, because MedHallu supplies ground-truth hallucinated / non-hallucinated pairs that the detection-F1 metric needs.

The short answer, supported by the numbers below, is: yes, the debate structure helps a lot; RL training helps further but only when deployment prompts match training prompts; shared LoRA adapters between the debate agents do not work; separate adapters with an iterated-best-response schedule do.

7.2 Agent Architecture and Logic

7.2.1 Claim Extractor

A light-weight stage that splits the draft answer into atomic claims and labels each as *material* (a clinical fact that needs evidence) or *contextual* (background that does not change the verdict).

7.2.2 Verifier (Agent A)

Evaluates each material claim under strict rules: the claim is `SUPPORTED` only if the evidence explicitly states the same fact with matching specifics (drug name, dose, diagnosis, value). When the evidence is ambiguous, the Verifier flags.

7.2.3 Challenger (Agent B)

Attacks every `SUPPORTED` claim using a fixed challenge taxonomy: `CONTRAINDICATION`, `DOSAGE_CHECK`, `INTERACTION`, `GUIDELINE_CURRENCY`, `GAP_FINDING`. The Challenger is required to issue a challenge; it is not allowed to agree with the Verifier.

7.2.4 Blind Judge

Scores each claim on a ternary scale (1.0 supported, 0.5 topic match but different specifics, 0.0 unsupported), without seeing Agent A's original confidence. Hiding Agent A's number prevents the Judge from anchoring on it.

7.2.5 Routing Decision

If any material claim scores 0.0, the query is `ABSTAINED`. If the aggregate is at or above 0.9, it is `APPROVED`. Otherwise `REVISED`. The debate runs for up to two cycles, with early exit when every material claim reaches ≥ 0.95 confidence.

7.3 Coordination and Reinforcement Learning

7.3.1 GRPO Baseline with Detection-Aligned Reward

The first training attempt used Group Relative Policy Optimization on the Verifier with a detection-aligned reward: $+1.0$ for correctly flagging a hallucinated answer, -1.0 for missing one, $+1.0$ for correctly approving a ground-truth answer, -0.5 for wrongly flagging one, plus small format bonuses. The asymmetry between -1.0 and -0.5 reflects a clinical prior that a missed hallucination is more dangerous than a false alarm. An earlier Brier-score calibration reward improved calibration but did not improve detection, which taught me that RL optimizes whatever you reward and not a proxy.

7.3.2 Why Independent GRPO Fails

Training each debate agent independently with its own reward produced individually stronger agents (Verifier eval reward moved from 0.221 to 0.705, Challenger from 0.132 to 0.218) but did almost nothing for the full-pipeline detection F1 (0.642 to 0.657). The agents improved in isolation and did not learn to complement each other.

7.3.3 MARL with Counterfactual Credit (C3)

The fix is a hybrid reward:

$$r^{(i)}_{\text{hybrid}} = \alpha\, r^{(i)}_{\text{individual}} + \beta\, r^{(i)}_{\text{counterfactual}} + \gamma\, r_{\text{shared}}$$

with $\alpha = 0.5$, $\beta = 0.3$, $\gamma = 0.2$. The counterfactual term measures how much each agent's output changes the pipeline verdict:

$$r^{(i)}_{\text{counterfactual}} = r_{\text{pipeline}}(\text{all agents}) - r_{\text{pipeline}}(\text{agent}_i \leftarrow \text{default})$$

where the default output is neutral (Verifier confidence 0.5, empty challenge list).

7.3.4 Separate LoRA Adapters and Iterated Best Response

Four experiments with a shared LoRA adapter between Verifier and Challenger failed near random accuracy, because updating one agent's LoRA weights overwrote what the other agent had learned. The working configuration uses a separate rank-8 LoRA adapter per agent (α=16, dropout 0.1,

targeting q/k/v/o/gate/up/down projections) and trains them under an iterated best-response schedule: Verifier to convergence with Challenger frozen, then Challenger to convergence with Verifier frozen, repeat. Warm-starting from the GRPO checkpoints rather than from scratch cuts training time substantially.

7.4 Real-Time Hallucination Inference

At inference, MAD debate runs inside the Phase I pipeline as a drop-in replacement for the single Critic. The debate agents' LoRA adapters are loaded at service start and attached to a shared Qwen 2.5 3B Instruct backbone, so the per-query cost is a few additional LLM calls (one per debate round, capped at two rounds) rather than additional models.

7.5 Model Evaluation and Performance Analysis

7.5.1 Evaluation Tracks

Track 1 Pipeline vs. frontier LLMs on 80 clinical queries across 10 MIMIC-IV patients. Metrics: Entity F1, Precision, Recall, Hallucination Rate, Abstention Accuracy.

Track 2 Retrieval quality on 72 queries. Metrics: MRR, NDCG@15, Recall@15.

Track 3 Hallucination detection on 1,000 MedHallu pairs. Four configurations: Single Critic, MAD Base, MAD+GRPO v3, MAD+MARL C3 v2. Metrics: Detection F1, Precision, Recall, with 10,000-sample bootstrap 95% CIs.

7.5.2 Track 1 Results: Pipeline vs. Frontier LLMs

Table 7.1: Track 1 – Ground-truth evaluation (10 patients, 80 queries). Backbone for the pipeline is MiniMax M2.5.

Metric	**RAG (Ours)**	Sonnet 4.5	Haiku 3.5	GPT-5	GLM 4.6	Gemini 3 Pro
Entity F1	**0.639**	0.531	0.482	0.331	0.308	0.234
Entity Precision	**0.850**	0.763	0.796	0.556	0.524	0.627
Entity Recall	**0.555**	0.449	0.397	0.281	0.259	0.184
Semantic Sim.	**0.609**	0.563	0.586	0.471	0.395	0.454
ROUGE-L	0.167	0.157	0.159	**0.195**	0.176	0.117
Hallucination Rate	0.150	0.209	0.133	0.130	**0.062**	0.230
Abstention Acc.	**100%**	100%	100%	100%	90%	100%

Table 7.2: Track 1 alternate backbone – MiniMax M2.5 RAG specifically.

Metric	**RAG (Ours)**	GLM 4.6	GPT-5	Gemini 3 Pro	Haiku 3.5	Sonnet 4.5
Entity F1	**0.6711**	0.1981	0.2251	0.1539	0.3791	0.4489
Entity Precision	**0.8707**	0.2792	0.3499	0.5516	0.6542	0.6567
Entity Recall	**0.5913**	0.1756	0.1997	0.1276	0.3076	0.3731
ROUGE-L	0.1784	0.1956	**0.1909**	0.0839	0.1372	0.1567
Semantic Sim.	**0.6236**	0.3296	0.3636	0.3851	0.5387	0.5321
Hallucination Rate	0.1293	**0.0708**	0.1739	0.2103	0.1553	0.2480
Abstention Acc.	**100%**	33.3%	100%	100%	100%	100%

A few observations. The pipeline wins on Entity F1, Precision, and Recall against every baseline. ROUGE-L is the one metric where a baseline beats it; GPT-5's answers are longer and share more surface tokens with the reference, which is what ROUGE-L rewards, and is one of the reasons ROUGE-L is a poor proxy for clinical correctness. GLM 4.6 achieves the lowest hallucination rate, but that is because its recall is the lowest — it hallucinates little because it says little. The abstention-accuracy column captures this: GLM correctly abstains on only 90% of the truly unanswerable queries in Track 1 and 33.3% on the MiniMax-backbone Track 1 variant, meaning it sometimes answers questions it should have refused.

7.5.3 Results by Query Type

Breaking the evaluation down by query type shows where the pipeline wins most and where it is still beaten.

Table 7.3: Entity F1 by query type.

Type	**RAG (Ours)**	Sonnet 4.5	Haiku 3.5	GPT-5
FACTUAL	**0.607**	0.512	0.472	0.377
MEDICATION	**0.770**	0.383	0.522	0.007
TEMPORAL	0.721	**0.759**	0.686	0.660
SUMMARY	**0.550**	0.526	0.280	0.141

Medication is the biggest win (0.770 versus 0.007 for GPT-5), and it is worth unpacking why. When GPT-5 sees the full chart in a one-shot setting, it encounters `MedicationRequest` resources with `status: completed` fields. It over-interprets these and decides that it cannot reliably report any active medications, so it either hedges or refuses. The pipeline, by contrast, retrieves the MedicationRequest chunks with their status and date metadata, and the Reasoning agent is prompted to answer from those chunks directly. Focused retrieval beats "more capable" here — the bigger model over-reasons and produces "insufficient data"; the smaller model with the right evidence answers correctly.

Temporal is the one type where a baseline edges out the pipeline (Sonnet 4.5 at 0.759 versus 0.721). Sonnet 4.5 does relatively well on chronological reasoning when the events are explicit in the chart. This is an area for Phase III work on the temporal validator.

7.5.4 Track 2 Results: Embedding Fine-Tuning

Table 7.4: Track 2 – Retrieval quality (72 queries).

Model	MRR	NDCG@15	Recall@15
Base PubMedBERT	0.623	0.676	1.000
Fine-tuned	**0.914**	**0.934**	1.000
Improvement	+46.7%	+38.2%	—

Fine-tuning PubMedBERT on 760 clinical triplets with MultipleNegativesRankingLoss raises MRR from 0.623 to 0.914. Recall@15 was already at 1.0, which means the right chunks were always in the retrieved set; what fine-tuning did was move them up the ranking so the Reasoning agent sees them first. TripletLoss failed on the same data, and in hindsight the reason is simple: a margin-based loss needs positives and hard negatives that are far enough apart for the margin to be meaningful, which is exactly what clinical chunks about the same patient and topic are not.

7.5.5 Track 3 Results: Hallucination Detection

Table 7.5: Track 3 – Full-pipeline detection (1,000 MedHallu pairs).

Config	F1	95% CI	Prec.	Recall
Single Critic	0.551	[0.522, 0.578]	0.679	0.463
MAD Base	**0.645**	[0.622, 0.666]	0.545	0.789
MAD + GRPO v3	0.642	[0.620, 0.664]	0.539	0.796
MAD + MARL C3 v2	0.643	[0.621, 0.665]	0.537	0.802

Table 7.6: Track 3 – Matched-prompt detection (1,000 MedHallu pairs).

Config	F1	95% CI	Prec.	Recall
Single Critic	0.080	[0.058, 0.104]	0.894	0.042
MAD Base	0.357	[0.325, 0.388]	0.528	0.269
MAD + GRPO v3	0.393	[0.363, 0.423]	0.521	0.315
MAD + MARL C3 v2	**0.752**	[0.732, 0.772]	0.711	0.799

Under matched prompts, MARL C3 v2 outperforms every other configuration with non-overlapping confidence intervals. The key number is the gap between the full-pipeline column (0.643) and the

matched-prompt column (0.752) for MAD + MARL C3 v2, which is the signature of the prompt-distribution-mismatch problem discussed in §9.2.2.

Table 7.7: MARL C3 v2 IBR training progression.

Iteration	Acc.	F1	Prec.	Recall
Warm-start (GRPO v3)	47.0%	0.384	0.559	0.292
Iter 1	60.5%	0.599	0.702	0.522
Iter 2	72.0%	0.769	0.721	0.823
Iter 3	**82.0%**	**0.847**	0.813	**0.885**

Table 7.8: MARL ablation study – why shared LoRA fails.

Experiment	LoRA	Start	Accuracy	Status
MARL v1	Shared	Scratch	49.0%	Failed
MARL v2	Shared	Scratch	43.0%	Cancelled
MARL C3 v1	Shared	Scratch	48.5%	Failed
MARL Full Pipeline	Shared	Scratch	—	No improvement
MARL C3 v2	**Separate**	GRPO v3	**82.0%**	**Success**

Every IBR iteration improves monotonically, and every shared-LoRA configuration fails. Those two observations together are the cleanest takeaway from Phase II.

7.6 Data Processing Pipeline for Phase II

Two thousand MedHallu trajectories (1,000 hallucinated, 1,000 ground-truth) were preprocessed into the same chunk schema as the Phase I retrieval pipeline, split 1,800 / 200 train/eval. For each trajectory GRPO sampled $K = 4$ completions and computed group-relative advantages $A_i = (r_i - \bar{r})/(\sigma_r + \epsilon)$.

7.7 System Testing and Validation

7.7.1 Functionality Test Cases

ID	Scenario	Expected Result	Status
TC-01	Medication query with active prescription	`APPROVED` with MedicationRequest citation	Pass
TC-02	Medication query with fabricated dose	`ABSTAINED`; Numeric Validator flags	Pass
TC-03	Temporal query spanning 30-day window	Temporal Validator confirms ordering	Pass
TC-04	Numeric query with unit mismatch (mg/dL vs mmol/L)	Numeric Validator flags; `REVISED`	Pass
TC-05	Empty retrieval (no matching chunks)	`ABSTAINED`; no LLM call beyond Router	Pass
TC-06	Cross-patient leakage attempt	403 from patient-scope middleware	Pass
TC-07	Audit chain tampered mid-session	Integrity Verifier rejects on `GET /audit`	Pass

Table 7.9: Functionality test cases.

8 System Deployment and Infrastructure

8.1 Overview

EHR-Copilot runs as a small collection of containerized services: a FastAPI backend, an embedding and retrieval service (CPU or GPU), an Ollama LLM runtime (GPU), a SQLite database for the audit log, and the Streamlit UI. In development, everything runs on a single GPU workstation. In production, the GPU-bound services would move to a Kubernetes cluster with a GPU-backed node pool.

8.2 Frontend Application

8.2.1 Technology Stack

Streamlit 1.40+, custom components for the citation drawer and timeline, Tailwind for styling where Streamlit allows overrides.

8.2.2 Core Features

Chat-style query submission, claim-level citation rendering, click-to-verify source drawer, per-query timeline view, and a dedicated abstention banner with the Critic's reason. The UI distinguishes "I checked and could not find evidence" from "I checked and found contradictory evidence," because those two states mean different things to a clinician.

8.2.3 Frontend-to-Backend Interaction

Queries are submitted via JSON POST to `/query`. Streaming responses use server-sent events so the reviewer sees the pipeline stages progress (Router, Retrieval, Reasoning, Validators, Critic) rather than waiting on a black-box spinner.

8.3 Backend API Layer

8.3.1 Technology Stack

Python 3.11, FastAPI 0.115+, Uvicorn, Pydantic v2 for schema validation, sentence-transformers 3.0+ for the embedding path, faiss-cpu 1.8+, bm25s 0.2+, `fhir.resources` 8.0+ for FHIR parsing, `medspacy` 1.2+ for note segmentation, `rapidfuzz` 3.8+ for the citation engine, `python-dateutil` and `pint`/`ucumvert` for the validators.

8.3.2 Key Endpoints

`POST /query`, `POST /patient/load`, `GET /audit/{session_id}`, `GET /health`. Middleware enforces Request-ID, timing, and patient-scope.

8.3.3 Real-Time Inference

End-to-end, the pipeline makes between three and four LLM calls (Router, Reasoning, Critic, optionally one revision). Average latency at Phase I evaluation was around 104 seconds per query on the MiniMax M2.5 backbone. That is too slow for real-time bedside use and appropriate for chart review, care planning, and clinical research.

8.4 Authentication and Access Control

Role-based access control distinguishes reviewers, admins, and auditors. Each query carries the submitting user's ID; the audit log preserves it. PHI never enters logs beyond the hash-chained audit table, which itself is access-controlled.

8.5 Audit Database (SQLite)

The audit log is stored in a single-file SQLite database with `aiosqlite` for async access. Each row is an append-only record containing the SHA-256 hash of the previous row, which makes the chain tamper-evident: any modification anywhere in the chain invalidates every hash that follows.

8.6 System Integration Overview

In development, docker-compose ties the services together. In production, each service is a separate Kubernetes Deployment; the LLM runtime lives on a GPU node pool with a taint so only the inference pods schedule there; model checkpoints are loaded at pod start and pinned in GPU memory to avoid cold-start latency on the first query.

8.7 Version Control and GitHub Usage

The repository is organized by service: `backend/`, `retrieval/`, `training/` (Phase II), `frontend/`, `eval/`, `infra/`. Training runs are reproducible from a pinned YAML config and a checkpointed dataset snapshot.

8.8 Hosting and Deployment

The intended deployment target is a self-hosted Kubernetes cluster with GPU nodes, which is the configuration most compatible with HIPAA data-residency requirements. A BAA-covered cloud

deployment is possible but not the default.

8.9 Security and Scalability Considerations

Horizontal scaling is limited by LLM inference cost rather than the retrieval or orchestration layer. Caching of identical queries is implemented but disabled by default — for a clinical assistant, one cache hit returning stale data is worse than one cold query returning fresh data. Rate limiting is handled at the ingress.

9 Results, Challenges, and Future Work

9.1 System Results and Performance Overview

The headline results, pulled together: Entity F1 = 0.639 across 80 clinical queries on 10 MIMIC-IV patients, 20% better than the strongest one-shot frontier baseline (Claude Sonnet 4.5 at 0.531). Precision at 0.850. Abstention accuracy 100%. On the MiniMax M2.5 backbone specifically, Entity F1 reaches 0.671. Medication queries land at 0.770 vs. GPT-5's 0.007. The Phase II debate layer improves Detection F1 on MedHallu from 0.551 to 0.645 (+17%) with no training, and MARL with separate LoRA adapters reaches 0.752 on matched prompts.

What I want to emphasize is that the largest single improvement in the system comes from an *architectural* change, not from training: moving from a single-critic verifier to a three-agent structured debate adds roughly ten points of Detection F1 without any RL at all. RL on top of that helps — but it is an amplifier on a good architecture, not a substitute for one.

9.2 Challenges Faced During Development

9.2.1 Data: Small Footprint, Edge Cases Everywhere

The MIMIC-IV FHIR demo is a small slice of MIMIC, and the query coverage I could hand-author against ten patients is necessarily limited. Synthea bundles filled some of the gap but have their own artifacts (too-clean timelines, unrealistic medication adherence). Several bugs in the FHIR parser were only caught when a Synthea bundle surfaced them, which I only noticed because the MIMIC data happened not to exercise the same code path.

9.2.2 Prompt Distribution Mismatch

The single hardest problem in Phase II was that the GRPO-trained Verifier improvements did not transfer into the full debate pipeline. The reason, once I figured it out, is blunt: during training the Verifier saw a short uniform prompt of the form "Answer: {...}; Evidence: {...}; return JSON verdict". At inference inside the debate engine it saw a much longer, structured prompt with claim-by-claim verification, structured output fields, and multi-claim JSON arrays. The two prompts occupy different regions of the model's input distribution, and the LoRA fine-tuning is effectively invisible through the debate engine because the attention patterns the adapter learned to modulate are not the ones at play. The effect is similar to the brittleness InstructGPT and code-generation RLHF work has documented. I treat it as an open problem for applying RL to multi-stage NLP pipelines and sketch remedies in the future-work section.

9.2.3 Shared LoRA Adapters Do Not Work for Cooperating Agents

Four MARL attempts with a shared LoRA between the Verifier and the Challenger all collapsed to near-random accuracy. Training one agent's weights silently overwrote what the other agent had learned. The fix — a separate rank-8 adapter per agent, trained under iterated best response — was discovered only after systematically ruling out reward design, hyperparameters, and warm-start versus scratch. In retrospect it is obvious; in the moment it was not.

9.2.4 Reward Design Is Not Subtle

An earlier reward based on Brier-score calibration improved confidence calibration but did not improve detection accuracy. RL optimizes exactly what you reward, and a proxy reward will give you the proxy, not the thing you care about. The detection-aligned reward with the asymmetric $-1.0/-0.5$ penalty structure replaced it and gave the actual target metric gains.

9.2.5 End-to-End Integration and Schema Drift

A surprising fraction of Phase I time went to making five services agree on what a debate trace, a claim, and a citation-mapping record look like. Most of the integration bugs were schema-drift bugs, where one service's notion of a claim lagged another's by one field.

9.3 Lessons Learned

A few things I would carry into adjacent work:

- **Architecture before training.** A five-stage pipeline with a principled abstention rule beats a bigger one-shot model on this task. RL improvements on top are real but smaller than the architectural move, and they are brittle to prompt format.
- **Match training and deployment prompts.** An RL-fine-tuned adapter is only active when the input distribution it was fine-tuned on is the one it sees at inference. If those differ, the adapter may as well not be loaded.
- **Separate parameters for cooperating agents.** Shared LoRA between a Verifier and a Challenger is not worth trying; budget a second adapter from the start.
- **Retrieval pays back disproportionately.** A well-tuned hybrid retriever with a reranker gives more than RL on the Reasoning agent ever did. Spend time there first.
- **Audit is a first-class product requirement.** The hash-chained log made me much more confident in my own results during Phase I, not only in compliance contexts.

9.4 Future Work

9.4.1 Matched-Prompt Training Trajectories

The direct fix for the prompt-distribution-mismatch problem is to generate training trajectories from the actual pipeline prompts, not from a simpler training-only prompt. The cost is that each training trajectory now requires a full debate run rather than a single decoder pass, which roughly multiplies the compute budget by the number of agents in the debate.

9.4.2 Full MIMIC-IV Integration

Moving from the demo to the full MIMIC-IV release stresses every layer: FAISS index size, embedding throughput on the hot path, audit-log storage, and the Reasoning agent's ability to maintain quality when the retrieval pool is twenty times larger.

9.4.3 Larger Backbones and Different PEFT Methods

The debate agents currently run on Qwen 2.5 3B Instruct because it is tractable on academic hardware. Whether 7B or 14B models exhibit different MARL dynamics — faster IBR convergence, less sensitivity to prompt format — is open. Alternative PEFT methods (IA^3, DoRA) may also change the shared-parameter-space story.

9.4.4 Better Temporal Reasoning

Temporal queries are the one category where a baseline (Sonnet 4.5) beats the pipeline. A richer Temporal Validator that reasons over the patient's timeline as a graph — rather than as a set of dates — is the obvious next step.

9.4.5 Clinician-Facing UI Work

The Streamlit UI was a development tool. A clinician-facing surface needs a verdict-first answer view, a provenance drawer that opens the underlying note in a side panel, and inline explanations for abstentions that a clinician can act on.

9.4.6 Operational Resilience

Graceful degradation paths — cached retrieval when the embedding service is down, deterministic-only verification when the Critic LLM is down, a single-agent fallback when the debate engine is unavailable — are not yet built. They are a prerequisite for any real clinical deployment.

9.5

EHR-Copilot is, in the end, a small-pipeline answer to a large-model problem. The combination of focused retrieval, specialized agents, deterministic validators, and a principled abstention rule outperforms every one-shot frontier baseline on patient-specific clinical QA in this evaluation. The Phase II debate and MARL extensions are a useful second layer and, equally useful, a clear warning about prompt-distribution mismatch when RL is applied to multi-stage pipelines. Both results are worth carrying forward.

10 Conclusion

This project built and evaluated EHR-Copilot, a five-stage multi-agent RAG pipeline for faithful clinical question answering over FHIR electronic health records, together with a Phase II research extension into multi-agent debate and cooperative reinforcement learning.

The core findings:

1. The five-stage pipeline reaches Entity F1 = 0.639 on 80 clinician-style queries over 10 MIMIC-IV patients, a 20% improvement over the best one-shot frontier baseline. Precision is 0.850, abstention accuracy is 100%, and the medication-query improvement over GPT-5 is two orders of magnitude.

2. Architecture matters more than scale for this task. A small cooperating pipeline with principled abstention beats larger models doing everything in one pass.

3. On the Phase II MedHallu benchmark, structured multi-agent debate raises Detection F1 from 0.551 to 0.645 with no training. Cooperative MARL with separate LoRA adapters and iterated best response reaches 0.752 on matched prompts. RL improvements with mismatched prompts mostly vanish, which identifies prompt-distribution alignment as a general challenge for RL in multi-stage NLP pipelines.

4. Privacy (patient-scoped ephemeral indexes, zero cross-patient leakage by construction) and auditability (SHA-256 hash-chained logs) are feasible to build into a clinical RAG pipeline from day one, and in my experience, doing so improved confidence in my own results during development, not only in a compliance context.

The most useful direction for follow-on work is generating training trajectories from the actual pipeline prompts to close the prompt-distribution-mismatch gap, followed by full MIMIC-IV integration and a proper clinician-facing UI.

References

[1] G. Xiong et al., "Benchmarking Retrieval-Augmented Generation for Medicine," *Findings of ACL*, 2024.

[2] Q. Wu et al., "AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation," *arXiv:2308.08155*, 2023.

[3] X. Tang et al., "MedAgents: Large Language Models as Collaborators for Zero-shot Medical Reasoning," *arXiv:2311.10537*, 2024.

[4] Z. Ji et al., "Survey of Hallucination in Natural Language Generation," *ACM Computing Surveys*, vol. 55, no. 12, 2023.

[5] A. Johnson et al., "MIMIC-IV Clinical Database Demo on FHIR," *PhysioNet*, 2023.

[6] C. Zakka et al., "Almanac: Retrieval-Augmented Language Models for Clinical Medicine," *NEJM AI*, 2024.

[7] S. Robertson and H. Zaragoza, "The Probabilistic Relevance Framework: BM25 and Beyond," *Foundations and Trends in Information Retrieval*, vol. 3, no. 4, pp. 333–389, 2009.

[8] Y. Gu et al., "Domain-Specific Language Model Pretraining for Biomedical Natural Language Processing," *ACM CHIL*, 2021.

[9] E. J. Hu et al., "LoRA: Low-Rank Adaptation of Large Language Models," *ICLR*, 2022.

[10] L. Du et al., "Multi-Agent Debate for Medical Text Verification," *EMNLP*, 2024.

[11] S. Liang et al., "Encouraging Divergent Thinking in Large Language Models through Multi-Agent Debate," *arXiv:2305.19118*, 2023.

[12] S. Yan et al., "Corrective Retrieval Augmented Generation," *arXiv:2401.15884*, 2024.

[13] Z. Wang et al., "MMOA-RAG: Multi-Agent Orchestration for RAG," *arXiv:2024*, 2024.

[14] UTAustin-AIHealth, "MedHallu: A Medical Hallucination Benchmark," *HuggingFace Datasets*, 2024.

[15] DeepSeek-AI, "DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning," *arXiv:2501.12948*, 2025.

[16] J. Chen et al., "HuatuoGPT-o1: Medical Complex Reasoning with Verifiable RL," *arXiv:2412.18925*, 2024.

[17] L. Ouyang et al., "Training Language Models to Follow Instructions with Human Feedback," *NeurIPS*, 2022.

[18] Y. Li et al., "Competition-Level Code Generation with AlphaCode," *Science*, vol. 378, no. 6624, 2022.

A Appendix

A.1 Example FHIR Query Patterns Used by the Entity Verifier

Listing A.1: Representative FHIR lookups for validator checks.

```
# Is this medication active for the patient right now?
GET /MedicationRequest?patient={pid}
    &status=active
    &code={rxnorm}

# Latest lab observation in a given LOINC code
GET /Observation?patient={pid}
    &code={loinc}
    &_sort=-date&_count=1

# All encounters in a date window
GET /Encounter?patient={pid}
    &date=ge{t0}&date=le{t1}
```

A.2 API Endpoint Sketch

Listing A.2: Core FastAPI routes.

```
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel

app = FastAPI()

class QueryIn(BaseModel):
    patient_id: str
    question: str

@app.post("/query")
def run_query(q: QueryIn):
    trace_id = orchestrator.submit(q.patient_id, q.question)
    return {"trace_id": trace_id}

@app.get("/audit/{session_id}")
def audit(session_id: str):
    chain = audit_log.chain_for(session_id)
    if not chain.verify():
```

```
        raise HTTPException(409, "audit chain integrity failure")
    return chain.entries
```

A.3 Router Agent Prompt Template

Listing A.3: Router prompt (abridged, Jinja2).

```
You classify clinical questions into a query type.

Types:
- TEMPORAL          (dates, trends, ordering)
- NUMERIC           (lab values, dosages, arithmetic)
- TEMPORAL_NUMERIC  (both)
- FACTUAL           (problem list, conditions)
- MEDICATION        (current or past prescriptions)
- SUMMARY           (narrative overview)
- REASONING         (cause/effect, hypothesis)

Return ONLY JSON:
{"type": "...", "time_window": {...} | null}
```

A.4 LoRA Config (Phase II)

Listing A.4: PEFT LoRA config used per debate agent.

```
from peft import LoraConfig

lora_cfg = LoraConfig(
    r=8,
    lora_alpha=16,
    lora_dropout=0.1,
    target_modules=["q_proj","k_proj","v_proj","o_proj",
                    "gate_proj","up_proj","down_proj"],
    task_type="CAUSAL_LM",
)
```

A.5 User Interface Snapshots